JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface ScheduledExecutorService

All Superinterfaces:

`AutoCloseable`, `Executor`, `ExecutorService`

All Known Implementing Classes:

`ForkJoinPool`, `ScheduledThreadPoolExecutor`

```
public interface ScheduledExecutorService
extends ExecutorService
```

An `ExecutorService` that can schedule commands to run after a given delay, or to execute periodically.

The `schedule` methods create tasks with various delays and return `ScheduledFuture` objects that can be used to cancel or check execution. When delays elapse, tasks are enabled for execution and behave in accord with other `ExecutorService` tasks, except that `scheduleAtFixedRate` and `scheduleWithFixedDelay` methods create and execute tasks that run periodically until cancelled.

Commands submitted using the `Executor.execute(Runnable)` and `ExecutorService` `submit` methods are scheduled with a requested delay of zero. Zero and negative delays (but not periods) are also allowed in `schedule` methods, and are treated as requests for immediate execution.

All `schedule` methods accept *relative* delays and periods as arguments, not absolute times or dates. It is a simple matter to transform an absolute time represented as a `Date` to the required form. For example, to schedule at a certain future `date`, you can use: `schedule(task, date.getTime() - System.currentTimeMillis(), TimeUnit.MILLISECONDS)`. Beware however that expiration of a relative delay need not coincide with the current `Date` at which the task is enabled due to network time synchronization protocols, clock drift, or other factors.

The `Executors` class provides convenient factory methods for the ScheduledExecutorService implementations provided in this package.

Usage Example

Here is a class with a method that sets up a ScheduledExecutorService to beep every ten seconds for an hour:

```
import static java.util.concurrent.TimeUnit.*;
class BeeperControl {
  private final ScheduledExecutorService scheduler =
```

```
    Executors.newScheduledThreadPool(1);

  public void beepForAnHour() {
    Runnable beeper = () -> System.out.println("beep");
    ScheduledFuture<?> beeperHandle =
      scheduler.scheduleAtFixedRate(beeper, 10, 10, SECONDS);
    Runnable canceller = () -> beeperHandle.cancel(false);
    scheduler.schedule(canceller, 1, HOURS);
  }
}
```

Since:

1.5

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
ScheduledFuture<?>	**schedule**(Runnable command, long delay, TimeUnit unit)	Submits a one-shot task that becomes enabled for execution after the given delay.
<V> ScheduledFuture<V>	**schedule**(Callable<V> callable, long delay, TimeUnit unit)	Submits a value-returning one-shot task that becomes enabled for execution after the given delay.
ScheduledFuture<?>	**scheduleAtFixedRate**(Runnable command, long initialDelay, long period, TimeUnit unit)	Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given period; that is, executions will commence after `initialDelay`, then `initialDelay + period`, then `initialDelay + 2 * period`, and so on.
ScheduledFuture<?>		

	scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)	Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next.

Methods declared in interface Executor

execute

Methods declared in interface ExecutorService

awaitTermination, close, invokeAll, invokeAll, invokeAny, invokeAny, isShutdown, isTerminated, shutdown, shutdownNow, submit, submit, submit

Method Details

schedule

```
ScheduledFuture<?> schedule(Runnable command,
                            long delay,
                            TimeUnit unit)
```

Submits a one-shot task that becomes enabled for execution after the given delay.

Parameters:

`command` - the task to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ScheduledFuture representing pending completion of the task and whose `get()` method will return `null` upon completion

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if command or unit is null

schedule

```
<V> ScheduledFuture<V> schedule(Callable<V> callable,
                                long delay,
                                TimeUnit unit)
```

Submits a value-returning one-shot task that becomes enabled for execution after the given delay.

Type Parameters:

`V` - the type of the callable's result

Parameters:

`callable` - the function to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ScheduledFuture that can be used to extract result or cancel

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if callable or unit is null

scheduleAtFixedRate

```
ScheduledFuture<?> scheduleAtFixedRate(Runnable command,
                                       long initialDelay,
                                       long period,
                                       TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given period; that is, executions will commence after `initialDelay`, then `initialDelay + period`, then `initialDelay + 2 * period`, and so on.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled via the returned future.
- The executor terminates, also resulting in task cancellation.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

If any execution of this task takes longer than its period, then subsequent executions may start late, but will not concurrently execute.

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`period` - the period between successive executions

`unit` - the time unit of the initialDelay and period parameters

Returns:

a ScheduledFuture representing pending completion of the series of repeated tasks. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if command or unit is null

`IllegalArgumentException` - if period less than or equal to zero

scheduleWithFixedDelay

```
ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                          long initialDelay,
                                          long delay,
                                          TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled via the returned future.
- The executor terminates, also resulting in task cancellation.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`delay` - the delay between the termination of one execution and the commencement of the next

`unit` - the time unit of the initialDelay and delay parameters

Returns:

a ScheduledFuture representing pending completion of the series of repeated tasks. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if command or unit is null

`IllegalArgumentException` - if delay less than or equal to zero

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.